Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Artificial Intelligence

ENCS3340

Project 1 Report

Prepared By:

Lara Foqaha	1220071	Section 4
Dana Taher	1221240	Section 1

Instructor:

Yazan Abu Farha

Date: May – 2025

1. Problem Formulation

In this project, we are solving a package delivery problem. Each package has a weight and a priority. We also have a fixed number of vehicles, and each one has a limited capacity. Our goal is to assign the packages to the vehicles in a way that reduces the total distance they travel as much as possible.
We describe the problem using the structure of a search problem, as learned in the lectures. Here's how we formulated it:

- **Initial State:**
  At the beginning, all the packages are at the shop (0, 0), and the vehicles are empty. No packages are assigned yet.

- **States:**
  A state is any valid way of assigning packages to vehicles and arranging their delivery order. The total weight in each vehicle should not go over its limit.

- **Actions:**
  An action means we move from one state to another by changing how the packages are assigned or changing the order in which they are delivered.

- **Goal Test:**
  A goal state is when:
  - Every package is assigned to a vehicle
  - No vehicle is overloaded
  - The total travel distance is as low as possible
  - Higher priority packages are delivered in a good way

- **Path Cost:**
  The cost is the total distance driven by all vehicles. We also add a small extra distance to the cost if a high priority package is not delivered efficiently.

- **State Space:**
  The state space includes all possible ways of assigning packages to the vehicles and arranging their routes. It is very large, so we can't check every possibility. That's why we use Simulated Annealing and Genetic Algorithm to find solutions faster.

2. Heuristics Used

To make our optimization more effective and realistic, we used several heuristics in both the Simulated Annealing (SA) and Genetic Algorithm (GA) implementations. These heuristics helped guide the search process and control how solutions are created and improved. We structured both algorithms based on the pseudo codes provided in the course textbook, and then implemented them step by step in Python.

- **Priority Assignment:**
  In the GA's initial population creation, we first sort the packages based on their priority. Packages with higher priority (lower number) are placed earlier. This way, more important packages get assigned to vehicles first before capacity fills up. We didn't sort the packages by priority in SA because the goal of Simulated Annealing is to explore the solution space more randomly at the beginning. SA works by starting from a completely random initial state and then gradually improving it through small controlled changes (neighbors).

- **Priority Based Distance Penalty:**
  In our route distance calculation, we apply a penalty based on the priority of each package. For example, a package with priority 1 has no penalty, but lower priority packages (priority 2, 3, …) increase the route distance slightly. The penalty is calculated like this:
  adjusted_distance = distance × (1.0 + 0.1 × (priority - 1))
  This means that delivering less important packages makes the total distance worse to prioritize more important ones.
  However, to avoid long distances, if the adjusted distance becomes 1.5 (a defined constant *PRIORITY_DISTANCE_TOLERANCE* which can be changed) times or more than the normal distance, we ignore the penalties and just use the base distance. This keeps the solution balanced.

- **Random Initial Assignment (SA):**
  In the SA algorithm, we start with a completely random assignment of packages to vehicles, while checking that each vehicle's capacity is not exceeded. If a package doesn't fit, we reshuffle and retry up to 500 times (a defined constant *MAX_ATTEMPTS* which can be changed) until a valid starting solution is found. This gives the algorithm enough variety without breaking constraints.

- **Neighbor Generation by Swapping Packages:**
  To explore new solutions in SA, we randomly swap one package from one vehicle with a package from another, as long as the new weights still fit within both vehicles' capacity. This method helps find better combinations without violating constraints. If we can't find a valid swap in 100 tries, we return a simple copy of the current state.

- **Crossover and Mutation in GA:**
  In the GA, each child solution is made by combining packages from two parents. After merging, we remove duplicate packages and **sort by priority**, then reassign packages to vehicles one by one without exceeding the capacity.
  Mutation is done by randomly swapping two packages between two vehicles, just like in SA but only if the swap is valid. This gives variety and helps escape local optima.

3. Handling Constraint Violation

In both algorithms, we had to make sure that no vehicle is ever overloaded beyond its capacity. This was the main constraint in the problem. In our code, we handled this by checking the total weight of packages before adding a new one to a vehicle. If adding the package would go over the vehicle's capacity, it was skipped.

For **Simulated Annealing**, during the initial solution creation, we used a loop that randomly assigns packages to vehicles, but only if their total weight after adding the new package stays within the allowed limit. If any package could not be placed without exceeding capacity, the attempt is marked as a failure and the algorithm tries again (up to 500 times). This retry loop ensures that we always start with a valid solution.

In the ***random_neighbor*** function of SA, we also make sure that any package swap between two vehicles does not cause capacity overflow. Before swapping two packages, we calculate the new total weight for both vehicles and only allow the swap if both remain within capacity.

For **Genetic Algorithm**, similar checks are done during population generation and when creating new children from parents (in the *assign_packages_randomly()* and *reproduce()* functions). Each package is only added to a vehicle if its weight fits. In the mutation step, swaps are also only allowed if the new weights stay within limits. If no valid solution can be created after all attempts, the program exits with an error message.

4. Parameter Tuning

For SA we used the following parameters:

- **Initial Temperature (*INITIAL_TEMP*)**: We set it to 1000. This value controls how likely the algorithm is to accept worse solutions at the start. A higher value helps explore more of the solution space at the beginning.

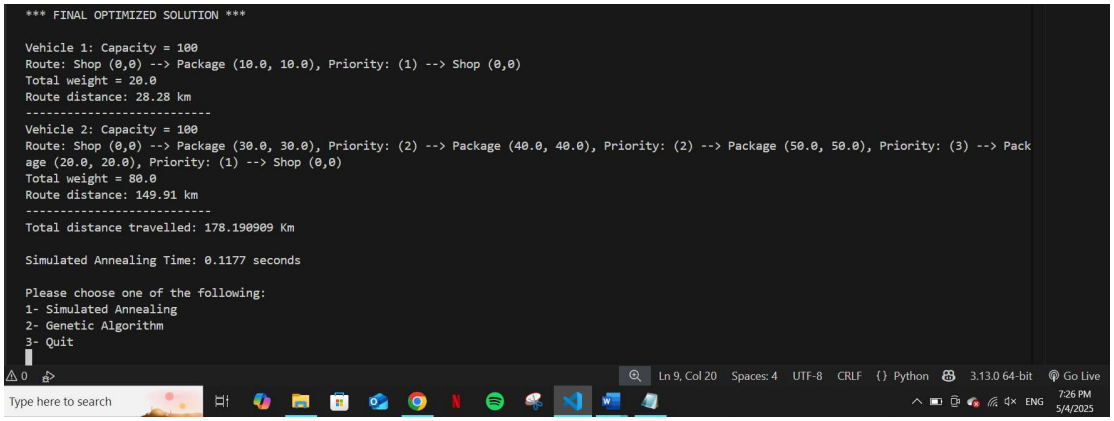
- **Cooling Rate (*COOLING_RATE*):** We used 0.95, meaning the temperature gets multiplied by 0.95 every round. We found this slow cooling helped improve results compared to faster drops like 0.75. In Figures below we first tried *COOLING_RATE* = 0.75 then changed it to 0.95. We observed that although the algorithm runs faster, it settles on worse solutions due to early convergence. Using 0.95 gave more consistent results.

```
*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (30.0, 30.0), Priority: (2) --> Package (50.0, 50.0), Priority: (3) --> Package (40.0, 40.0), Priority: (2) --> Shop
 (0,0)
Total weight = 60.0
Route distance: 152.74 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (20.0, 20.0), Priority: (1) --> Package (10.0, 10.0), Priority: (1) --> Shop (0,0)
Total weight = 40.0
Route distance: 56.57 km
---------------------------
Total distance travelled: 209.303607 Km

Simulated Annealing Time: 0.0239 seconds
```

Figure 1: SA results with 0.75 cooling rate



```
*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (10.0, 10.0), Priority: (1) --> Shop (0,0)
Total weight = 20.0
Route distance: 28.28 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (30.0, 30.0), Priority: (2) --> Package (40.0, 40.0), Priority: (2) --> Package (50.0, 50.0), Priority: (3) --> Pack
age (20.0, 20.0), Priority: (1) --> Shop (0,0)
Total weight = 80.0
Route distance: 149.91 km
---------------------------
Total distance travelled: 178.190909 Km

Simulated Annealing Time: 0.1177 seconds

Please choose one of the following:
1- Simulated Annealing
2- Genetic Algorithm
3- Quit
```

Figure 2: SA results with 0.95 cooling rate

- **Stop Temperature** (*STOP_TEMP*): We used 1 as the threshold to stop the algorithm. Once the temperature is too low, no more exploration is useful.

- **Iterations per temperature** (*ITERATIONS_PER_TEMP*): We set this to 100. This controls how many neighbors are checked before reducing the temperature. Trying more neighbors per level helped find better swaps.

For GA we used the following parameters:

- **Population Size** (*POP_SIZE*): We used 60 individuals. This gave enough diversity without making the algorithm too slow.
- **Generations** (*GENERATIONS*): We used 500 generations. Fewer generations dont give the algorithm enough time to improve. 500 was a good balance between time and solution quality.
- **Mutation Probability** (*MUTATION_PROB*): We set this to 0.1. This means there's a 10% chance of random mutation happening to a child. We tried values like 0.01, 0.05, and 0.1. The table below shows results for same input but different probabilities.

Metric \ Mutation probability	**0.01**	**0.05**	**0.1**
Best distance	405.32 km	401.95 km	401.95 km
Worst distance	539.2 km	417.73 km	402.13 km
Stability	inconsistent	Less consistent	Very consistent
Exploration	high	balanced	less (converges fast)

Table 1:GA results for different mutation probabilities

We found that a value of 0.1 gave the most consistent results, always reaching the best known distance of 401.95 km, while lower values like 0.01 led to unstable outcomes and higher worst case distances, indicating too much randomness in the search.

5. Test Cases

5.1 Case One: Basic Feasibility Test

The input file for this case is shown in Figure 3, where packages have the format:
X Y Weight Priority

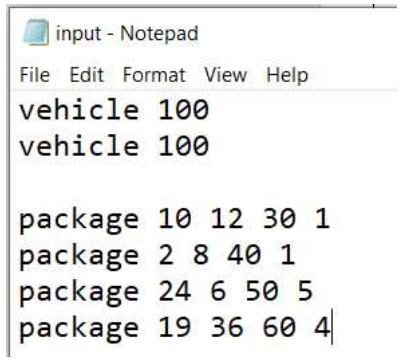


Figure 3: Test case 1 input file

All packages should be assigned to vehicles in a way that ensures no vehicle carries more than its capacity (100 kg in this case). The solution is valid only if no packages are left unassigned and the total weight per vehicle stays within its capacity. The following figures show valid results for both SA and GA.

```
*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (10.0, 12.0), Priority: (1) --> Package (19.0, 36.0), Priority: (4) --> Shop (0,0)
Total weight = 90.0
Route distance: 89.65 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (24.0, 6.0), Priority: (5) --> Package (2.0, 8.0), Priority: (1) --> Shop (0,0)
Total weight = 90.0
Route distance: 64.97 km
---------------------------
Total distance travelled: 154.619399 Km

Simulated Annealing Time: 0.1591 seconds
```

Figure 4: SA results for test case 1

```
*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (2.0, 8.0), Priority: (1) --> Package (19.0, 36.0), Priority: (4) --> Shop (0,0)
Total weight = 100.0
Route distance: 91.54 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (10.0, 12.0), Priority: (1) --> Package (24.0, 6.0), Priority: (5) --> Shop (0,0)
Total weight = 80.0
Route distance: 61.68 km
---------------------------
Total distance travelled: 153.219456 Km

Genetic Algorithm Time: 0.2024 seconds
```

Figure 5: GA results for test case 1

5.2 Case Two: Overcapacity Package

The input file for this case is shown in Figure 6 below:

```
input - Notepad
File  Edit  Format  View  Help
vehicle 100
vehicle 100

package 10 12 150 1
```

Figure 6: Test case 2 input file

The expected output is that the system should detect that the package exceeds all available vehicle capacities and should not assign it. Instead, it should handle the situation by notifying the user or logging the issue clearly. The figure below shows valid handling of overcapacity packages.

```
Enter the name of the input file:
input.txt
Read 1 packages and 2 vehicles

Please choose one of the following:
1- Simulated Annealing
2- Genetic Algorithm
3- Quit
1
Could not assign all packages within capacity limits.

PS C:\Users\HP\OneDrive\Desktop\Ai projects\project 1>
```

```
Enter the name of the input file:
input.txt
Read 1 packages and 2 vehicles

Please choose one of the following:
1- Simulated Annealing
2- Genetic Algorithm
3- Quit
2
Could not assign all packages within capacity limits.

PS C:\Users\HP\OneDrive\Desktop\Ai projects\project 1>
```

Figure 7: SA & GA results for test case 2

5.3 Case Three: Distance Optimization Test

The input file for this case is shown in Figure 8 below:

```
input - Notepad
File  Edit  Format  View  Help
vehicle 100
vehicle 100

package 2 3 20 1
package 10 12 30 2
package 5 7 25 1
package 8 9 10 3
package 15 4 15 2
package 12 6 20 1
```

Figure 8: Test case three input file

The system should assign and route packages in a way that minimizes the total distance travelled by both vehicles while respecting capacity limits. The test passes if all packages are assigned and the total route distance is less than or equal to the calculated optimal threshold.

```
*** INITIAL RANDOM SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (5.0, 7.0), Priority: (1) --> Package (12.0, 6.0), Priority: (1) --> Package (8.0, 9.0), Priority: (3) --> Package (2.0, 3.0), Priority: (1) -
-> Package (15.0, 4.0), Priority: (2) --> Shop(0,0)
Total weight = 90.0
Route Distance: 60.03 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (10.0, 12.0), Priority: (2) --> Shop(0,0)
Total weight = 30.0
Route Distance: 32.80 km
---------------------------

Initial total distance travelled: 92.828143 Km


--------------------------------------------------------

*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (5.0, 7.0), Priority: (1) --> Package (12.0, 6.0), Priority: (1) --> Package (8.0, 9.0), Priority: (3) --> Package (10.0, 12.0), Priority: (2)
 --> Package (15.0, 4.0), Priority: (2) --> Shop (0,0)
Total weight = 100.0
Route distance: 51.54 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (2.0, 3.0), Priority: (1) --> Shop (0,0)
Total weight = 20.0
Route distance: 7.21 km
---------------------------
Total distance travelled: 58.752156 Km

Simulated Annealing Time: 0.0069 seconds
```

Figure 9: AS results for test case 3

```
*** INITIAL RANDOM SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (12.0, 6.0), Priority: (1) --> Package (8.0, 9.0), Priority: (3) --> Shop(0,0)
Total weight = 30.0
Route Distance: 31.46 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (2.0, 3.0), Priority: (1) --> Package (5.0, 7.0), Priority: (1) --> Package (15.0, 4.0), Priority: (2) --> Package (10.0, 12.0), Priority: (2)
 --> Shop(0,0)
Total weight = 90.0
Route Distance: 46.09 km
---------------------------

Initial total distance travelled: 77.545769 Km


--------------------------------------------------------

*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (5.0, 7.0), Priority: (1) --> Package (12.0, 6.0), Priority: (1) --> Package (15.0, 4.0), Priority: (2) --> Package (10.0, 12.0), Priority: (2
) --> Package (8.0, 9.0), Priority: (3) --> Shop (0,0)
Total weight = 100.0
Route distance: 46.39 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (2.0, 3.0), Priority: (1) --> Shop (0,0)
Total weight = 20.0
Route distance: 7.21 km
---------------------------
Total distance travelled: 53.596237 Km

Genetic Algorithm Time: 0.6646 seconds
```

Figure 10: GA results for test case 3

The test passes as all packages were successfully assigned and the final total distance (53.59 km) for GA was significantly lower than the initial route distance (77.55 km), indicating effective optimization. Same thing for SA.

5.4 Test case Four: Simulated Annealing vs. Genetic Algorithm Comparison

The input file for this case is shown in Figure 11:

```
input - Notepad
File  Edit  Format  View  Help
vehicle 100
vehicle 100
vehicle 100

package 2 3 20 1
package 10 12 30 2
package 5 7 25 1
package 8 9 10 3
package 15 4 15 2
package 12 6 20 1
package 7 2 15 2
package 14 10 25 1
package 3 6 20 3
package 11 5 10 2
```

Figure 11: Test case 4 input file

Both algorithms are expected to generate valid solutions that meet all constraints, and their performance (measured by total distance and computation time) should be recorded to highlight their respective strengths and weaknesses. The results are shown below:

```
*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (8.0, 9.0), Priority: (3) --> Package (10.0, 12.0), Priority: (2) --> Package (12.0, 6.0), Priority: (1) --> Package (14.0, 10.0), Priority: (1)
 --> Package (15.0, 4.0), Priority: (2) --> Shop (0,0)
Total weight = 100.0
Route distance: 51.43 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (7.0, 2.0), Priority: (2) --> Package (11.0, 5.0), Priority: (2) --> Package (5.0, 7.0), Priority: (1) --> Package (3.0, 6.0), Priority: (3) -->
 Shop (0,0)
Total weight = 70.0
Route distance: 29.22 km
---------------------------
Vehicle 3: Capacity = 100
Route: Shop (0,0) --> Package (2.0, 3.0), Priority: (1) --> Shop (0,0)
Total weight = 20.0
Route distance: 7.21 km
---------------------------
Total distance travelled: 87.863189 Km

Simulated Annealing Time: 0.0083 seconds
```

figure 12: SA results for test case 4

```
*** FINAL OPTIMIZED SOLUTION ***

Vehicle 1: Capacity = 100
Route: Shop (0,0) --> Package (5.0, 7.0), Priority: (1) --> Package (14.0, 10.0), Priority: (1) --> Package (15.0, 4.0), Priority: (2) --> Package (12.0, 6.0), Priority: (1)
--> Package (11.0, 5.0), Priority: (2) --> Shop (0,0)
Total weight = 95.0
Route distance: 42.02 km
---------------------------
Vehicle 2: Capacity = 100
Route: Shop (0,0) --> Package (2.0, 3.0), Priority: (1) --> Package (7.0, 2.0), Priority: (2) --> Package (10.0, 12.0), Priority: (2) --> Package (8.0, 9.0), Priority: (3) --
> Package (3.0, 6.0), Priority: (3) --> Shop (0,0)
Total weight = 95.0
Route distance: 38.73 km
---------------------------
Vehicle 3: Capacity = 100
Route: Shop (0,0) --> Shop (0,0)
Total weight = 0
Route distance: 0.00 km
---------------------------
Total distance travelled: 80.755247 Km

Genetic Algorithm Time: 1.1116 seconds
```

Figure 13: GA results for test case 4

Both algorithms gave correct valid results, but the Genetic Algorithm found a shorter total distance but with more time, while Simulated Annealing was much faster in finishing the task, but gave a longer distance.